



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «13. Office communicator»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Тема: Основи проектування.

Мета роботи: Обрати зручну систему побудови UML-діаграм та навчитися будувати діаграми варіантів використання для системи що проектується, розробляти сценарії варіантів використання та будувати діаграми класів предметної області

Зміст

Теоретичні відомості.....	2
Хід виконання:.....	4
Завдання.....	4
Діаграма варіантів	5
Діаграма класів.....	9
Проектування БД.....	11
Вихідні коди класів системи	13
Контрольні запитання.....	16
Висновок	18

Теоретичні відомості

Мова UML є загальноцільовою мовою візуального моделювання, яка розроблена для специфікації, візуалізації, проектування та документування компонентів програмного забезпечення, бізнес-процесів та інших систем. Мова UML є досить строгим та потужним засобом моделювання, який може бути ефективно використаний для побудови концептуальних, логічних та графічних моделей складних систем різного цільового призначення. Ця мова увібрала в себе найкращі якості та досвід методів програмної інженерії, які з успіхом використовувалися протягом останніх років при моделюванні великих та складних систем.

Діаграма (diagram) – графічне уявлення сукупності елементів моделі у формі зв'язкового графа, вершинам і ребрам (дугам) якого приписується певна семантика. Нотація канонічних діаграм є основним засобом розробки моделей мовою UML.

Діаграма варіантів використання (Use-Cases Diagram) – це UML діаграма за допомогою якої у графічному вигляді можна зобразити вимоги до системи, що розробляється. Діаграма варіантів використання – це вихідна концептуальна модель проєктованої системи, вона не описує внутрішню побудову системи.

Діаграми варіантів використання є відправною точкою при збиранні вимог до програмного продукту та його реалізації. Дана модель будується на аналітичному етапі побудови програмного продукту (збір та аналіз вимог) і дозволяє бізнес-аналітикам отримати більш повне уявлення про необхідне програмне забезпечення та документувати його.

Діаграма варіантів використання складається з низки елементів. Основними елементами є: варіанти використання або прецеденти (use case), актор або дійова особа (actor) та відносини між акторами та варіантами використання (relationship).

Сценарії використання – це текстові уявлення тих процесів, які відбуваються при взаємодії користувачів системи та самої системи. Вони є чітко формалізованими, покроковими інструкціями, що описують той чи інший процес у термінах кроків досягнення мети. Сценарії використання однозначно визначають кінцевий результат.

Діаграми класів використовуються при моделюванні програмних систем найчастіше. Вони є однією із форм статичного опису системи з погляду її проєктування, показуючи її структуру [3]. Діаграма класів не відображає динамічної поведінки об'єктів зображених на ній класів. На діаграмах класів показуються класи, інтерфейси та відносини між ними.

Клас – це основний будівельний блок програмної системи. Це поняття є і в мовах програмування, тобто між класами UML та програмними класами є відповідність, що є основою для автоматичної генерації програмних кодів або для виконання реінжинірингу. Кожен клас має назву, атрибути та операції. Клас на діаграмі показується як прямокутник, розділений на 3 області. У верхній міститься назва класу, у середній – опис атрибутів

(властивостей), у нижній – назви операцій – послуг, що надаються об'єктами цього класу.

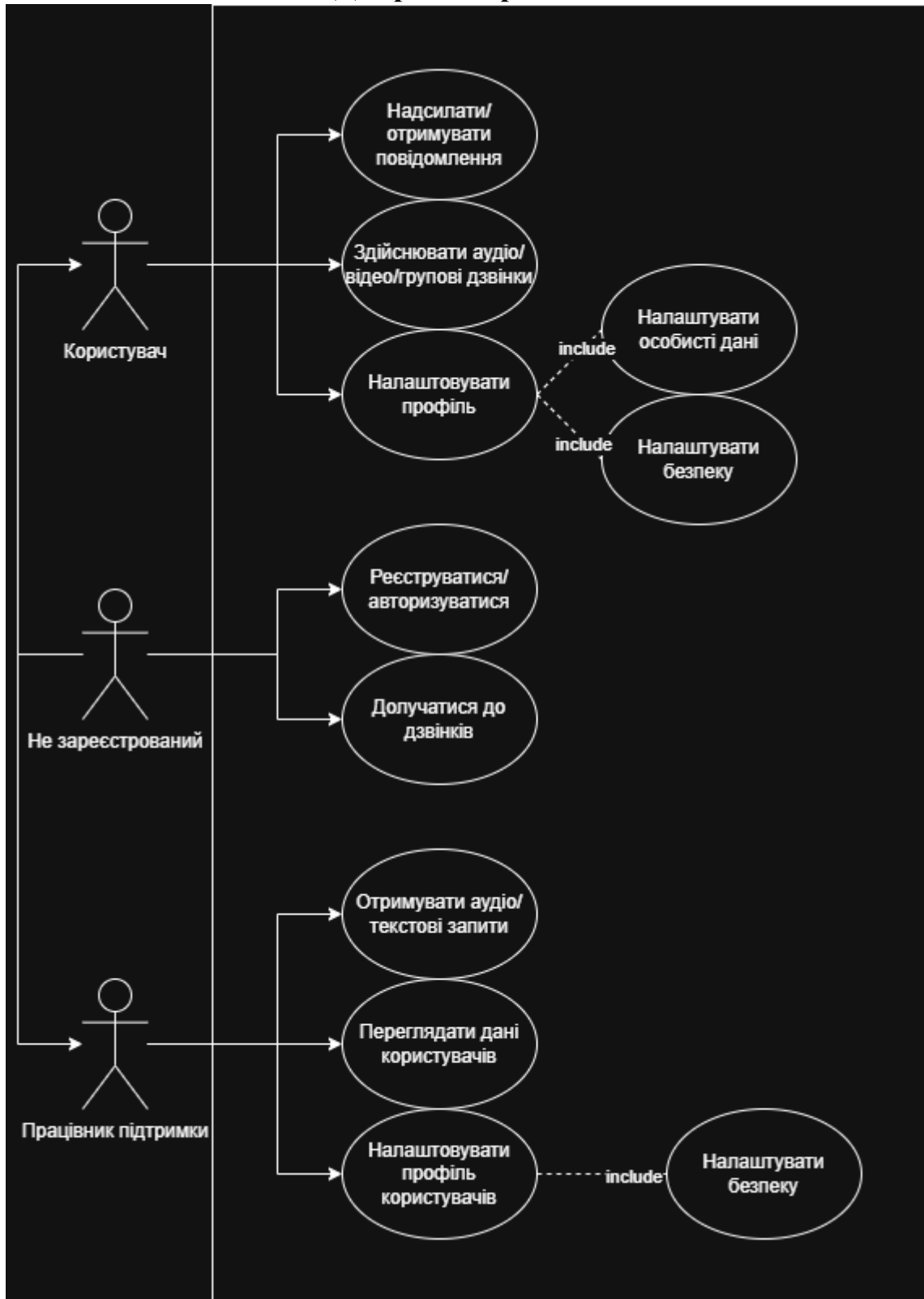
Логічна модель бази даних є структурою таблиць, уявлень, індексів та інших логічних елементів бази даних, що дозволяють власне програмування та використання бази даних. Процес створення логічної моделі бази даних зветься проєктування бази даних (database design). Проєктування відбувається у зв'язку з опрацюванням архітектури програмної системи, оскільки база даних створюється зберігання даних, одержуваних з програмних класів.

Хід виконання:

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Проаналізувати тему та спроектувати діаграму варіантів використання відповідно до обраної теми лабораторного циклу.
- Спроектувати діаграму класів предметної області.
- Вибрати 3 варіанти використання та написати за ними сценарії використання.
- На основі спроектованої діаграми класів предметної області розробити основні класи та структуру бази даних системи. Класи даних повинні реалізувати шаблон Repository для взаємодії з базою даних.
- Нарисувати діаграму класів для реалізованої частини системи.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму варіантів використання відповідно, діаграму класів системи, вихідні коди класів системи, а також зображення структури бази даних.

Діаграма варіантів



Сценарій 1: Надсилання повідомлення

Передумови:

Користувач авторизований у системі та має стабільне підключення до мережі.

Постумови:

Повідомлення доставлено адресату або збережено у черзі, якщо одержувач офлайн.

Взаємодіючі сторони:

Користувач, Система обміну повідомленнями (чат-сервер).

Короткий опис:

Користувач вводить текст повідомлення та надсилає його іншому користувачу або в групу. Система обробляє запит і доставляє повідомлення адресату.

Основний перебіг подій:

- Користувач відкриває чат.
- Користувач вводить повідомлення й натискає «Надіслати».
- Система приймає повідомлення та зберігає його.
- Система відправляє повідомлення одержувачу.
- Одержувач отримує повідомлення в реальному часі.

Винятки:

- №1: Втрата з'єднання. Повідомлення зберігається локально та надсилається пізніше.
- №2: Користувач недоступний (офлайн) — повідомлення потрапляє у чергу.
- №3: Невалідне повідомлення (порожнє або заборонений контент) — система відхиляє надсилання.

Примітки:

Можливе шифрування повідомлень для безпечної передачі.

Сценарій 2: Здійснення дзвінка**Передумови:**

Користувач авторизований і має дозволи на використання камери та мікрофона.

Постумови:

Відеодзвінок завершено, історія дзвінка збережена.

Взаємодіючі сторони:

Користувач (ініціатор), інший користувач (одержувач), Система відеодзвінків.

Короткий опис:

Користувач ініціює відеодзвінок до іншого користувача. Система встановлює з'єднання, передає аудіо- та відеопотоки між учасниками, а після завершення — фіксує результат.

Основний перебіг подій:

- Користувач відкриває список контактів.
- Обирає користувача та натискає «Відеодзвінок».
- Система надсилає виклик одержувачу.
- Одержувач приймає дзвінок.
- Система встановлює з'єднання й починає передачу потоків.
- Один із користувачів завершує дзвінок.
- Система припиняє передачу даних і зберігає історію.

Винятки:

- №1: Одержувач не відповідає — система повертає «Користувач недоступний».
- №2: Відсутній доступ до камери/мікрофона — дзвінок скасовується.
- №3: Проблеми зі з'єднанням — дзвінок автоматично завершується.

Примітки:

Система може знизити якість відео при низькій швидкості мережі.

Сценарій 3: Реєстрація нового користувача

Передумови:

Користувач має доступ до мережі та ще не має облікового запису.

Постумови:

Новий користувач зареєстрований і може входити до системи.

Взаємодіючі сторони:

Незареєстрований користувач, Система реєстрації.

Короткий опис:

Незареєстрований користувач створює обліковий запис, вводячи необхідні дані (логін, пароль, електронну адресу).

Основний перебіг подій:

- Користувач відкриває форму реєстрації.
- Вводить логін, пароль, e-mail.
- Система перевіряє коректність введених даних.
- Система створює новий обліковий запис.
- Користувач отримує повідомлення про успішну реєстрацію.

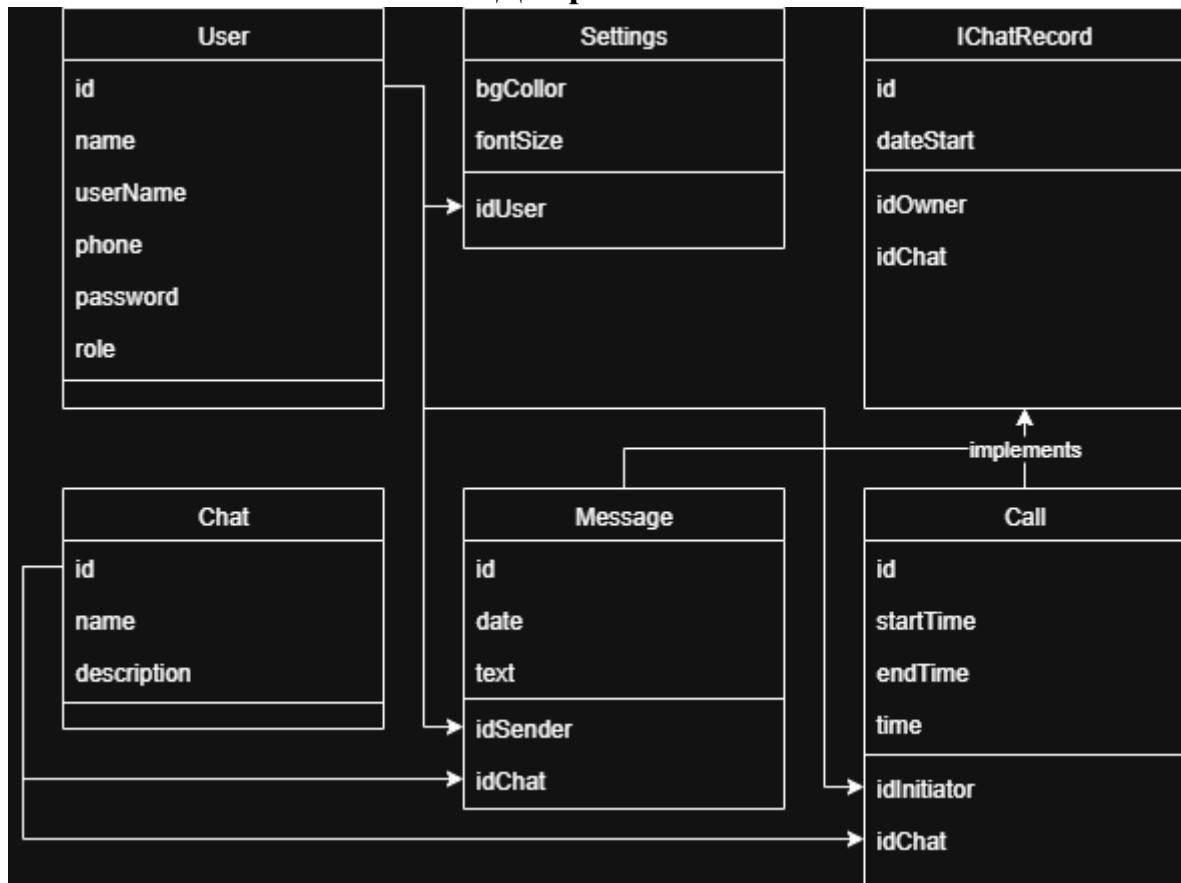
Винятки:

- №1: Користувач уже існує — система повідомляє про помилку.
- №2: Невалідні дані — реєстрація не відбувається.
- №3: Відсутній доступ до сервера — запит не обробляється.

Примітки:

Може бути передбачена активація через електронну пошту.

Діаграма класів



Опис

1. User – UserSettings

- **Тип відносин:** композиція
- **Пояснення:** Кожен користувач має власні налаштування інтерфейсу. Об'єкт `UserSettings` існує лише разом із відповідним користувачем – якщо користувача видалити, налаштування також зникають.

2. IChatRecord – Message

- **Тип відносин:** Узагальнення (`implements`)
- **Пояснення:** Клас `Message` реалізує інтерфейс `IRecordChat`, забезпечуючи поведінку запису в чаті для текстових повідомлень.

3. IChatRecord – Call

- **Тип відносин:** Узагальнення (`implements`)

- **Пояснення:** Клас Call реалізує інтерфейс IRecordChat, але містить специфічні атрибути для аудіо/відеодзвінків (час початку, тривалість тощо).

4. User – Message

- **Тип відносин:** асоціація
- **Пояснення:** Один користувач може надсилати багато повідомлень. Повідомлення має посилання на відправника через Id_Sender.

5. User – Call

- **Тип відносин:** асоціація
- **Пояснення:** Один користувач може ініціювати багато дзвінків. Поле Id_Initiator у Call вказує на користувача, який створив дзвінок.

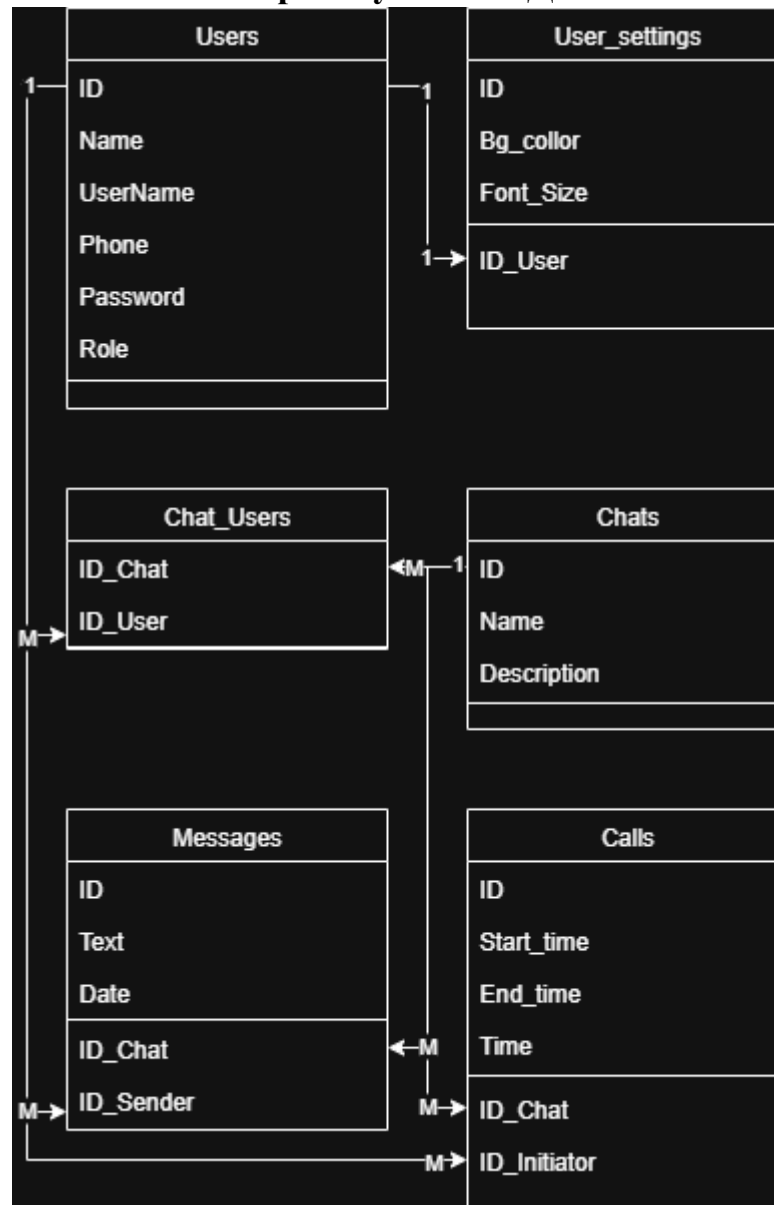
6. Chat – Message

- **Тип відносин:** композиція
- **Пояснення:** Повідомлення не існує без чату, до якого воно належить. Якщо чат видалено, його повідомлення також видаляються.

7. Chat – Call

- **Тип відносин:** композиція
- **Пояснення:** Кожен дзвінок належить певному чату. Без чату об'єкт дзвінка не має сенсу.

Проектування БД



Опис зв'язків між таблицями бази даних

1. Users – User_settings (1:1)

Між таблицями Users та User_settings існує зв'язок один до одного.

Кожен користувач має лише один набір налаштувань інтерфейсу, а кожні налаштування належать лише одному користувачу.

- **Users.ID ↔ User_settings.ID_User**

2. Users – Chats (M:N через Chat_Users)

Зв'язок між таблицями Users і Chats є багато до багатьох, реалізований через проміжну таблицю Chat_Users.

Це означає, що:

- один користувач може бути учасником багатьох чатів;
- один чат може містити багатьох користувачів.

А також, поля таблиця Chat_Users утворюють композитний первинний ключ:

- **ID_User** – зовнішній ключ на Users.ID;
- **ID_Chat** – зовнішній ключ на Chats.ID;

3. Chats – Messages (1:M)

Між таблицями Chats і Messages існує зв'язок один до багатьох.

Кожен чат може містити багато повідомлень, але кожне повідомлення належить лише одному чату.

- **Chats.ID ↔ Messages.ID_Chat**

4. Users – Messages (1:M)

Між таблицями Users і Messages існує зв'язок один до багатьох.

Користувач може надсилати багато повідомлень, але кожне повідомлення має лише одного відправника.

- **Users.ID ↔ Messages.ID_Sender**

5. Chats – Calls (1:M)

Між таблицями Chats і Calls існує зв'язок один до багатьох.

У межах одного чату може бути кілька дзвінків, але кожен дзвінок належить лише одному чату.

- **Chats.ID ↔ Calls.ID_Chat**

6. Users – Calls (1:M)

Між таблицями Users і Calls також існує зв'язок один до багатьох.

Користувач може ініціювати багато дзвінків, але кожен дзвінок має лише одного ініціатора.

- **Users.ID ↔ Calls.ID_Initiator**

Вихідні коди класів системи

```
namespace OfficeCommunicator
{
    public interface IRecordChat
    {
        int Id { get; set; }

        int ChatId { get; set; }

        DateTime Date { get; set; }

        void DisplayInfo();
    }

    public class User
    {
        public int Id { get; set; }

        public string Name { get; set; } = string.Empty;

        public string UserName { get; set; } = string.Empty;

        public string Phone { get; set; } = string.Empty;

        public string Password { get; set; } = string.Empty;

        public string Role { get; set; } = "User";

        public UserSettings Settings { get; set; }

        public List<Chat> Chats { get; set; } = new();

        public List<Message> Messages { get; set; } = new();

        public List<Call> Calls { get; set; } = new();

        public User()
        {
            Settings = new UserSettings { User = this };
        }
    }
}
```

// == Налаштування користувача ==

```
public class UserSettings
{
    public int Id { get; set; }

    public string BgColor { get; set; } = "#FFFFFF";

    public int FontSize { get; set; } = 14;

    public int UserId { get; set; }

    public User User { get; set; }
}
```

// == Клас чату ==

```
public class Chat
{
    public int Id { get; set; }

    public string Name { get; set; } = string.Empty;

    public string Description { get; set; } = string.Empty;

    public List<User> Participants { get; set; } = new();

    public List<IRecordChat> Records { get; set; } = new();

    public void AddRecord(IRecordChat record)
    {
        Records.Add(record);
    }
}
```

// == Повідомлення (реалізація IRecordChat) ==

```
public class Message : IRecordChat
{
    public int Id { get; set; }

    public string Text { get; set; } = string.Empty;

    public DateTime Date { get; set; } = DateTime.Now;
```

```

public int ChatId { get; set; }

public Chat Chat { get; set; }


public int SenderId { get; set; }

public User Sender { get; set; }


public void DisplayInfo()
{
    Console.WriteLine($"{Date}] {Sender?.UserName}: {Text}");
}
}


public class Call : IRecordChat
{
    public int Id { get; set; }

    public DateTime StartTime { get; set; } = DateTime.Now;

    public DateTime? EndTime { get; set; }

    public TimeSpan Duration => (EndTime ?? DateTime.Now) - StartTime;


    public int ChatId { get; set; }

    public Chat Chat { get; set; }


    public int InitiatorId { get; set; }

    public User Initiator { get; set; }


    public void DisplayInfo()
    {
        Console.WriteLine($"Call by {Initiator?.UserName} started at {StartTime}");
    }
}
}

```

Контрольні запитання

1. Що таке UML?

Універсальна мова моделювання (Unified Modeling Language) — стандарт для візуального опису структури та поведінки програмних систем.

2. Що таке діаграма класів UML?

Це діаграма, що показує класи системи, їх атрибути, методи та зв'язки між ними.

3. Які діаграми UML називають канонічними?

Діаграми, що входять до офіційного стандарту UML — наприклад: класів, варіантів використання, послідовності, діяльності, компонентів, станів тощо.

4. Що таке діаграма варіантів використання?

Діаграма, що показує взаємодію користувачів (акторів) із системою через її функції (use cases).

5. Що таке варіант використання?

Опис певного способу взаємодії користувача із системою для досягнення конкретної мети.

6. Які відношення можуть бути відображені на діаграмі використання?

Асоціація, включення (include), розширення (extend), узагальнення (генералізація акторів або варіантів).

7. Що таке сценарій?

Послідовність кроків, що описує, як саме виконується певний варіант використання.

8. Що таке діаграма класів?

Діаграма, яка показує структуру системи у вигляді класів, їхніх властивостей, методів та зв'язків.

9. Які зв'язки між класами ви знаєте?

Асоціація, агрегація, композиція, узагальнення (спадкування), залежність, реалізація інтерфейсу.

10. Чим відрізняється композиція від агрегації?

Композиція — жорсткий зв'язок «ціле—частина» (частина не існує без цілого). Агрегація — слабший зв'язок, частина може існувати окремо.

11. Чим відрізняються зв'язки типу агрегації від зв'язків композиції на діаграмах класів?

Агрегація позначається порожнім ромбом ($\diamond$), композиція — заповненим ($\blacklozenge$).

12. Що являють собою нормальні форми баз даних?

Це правила структуризації таблиць, які усувають дублювання даних і покращують цілісність БД.

13. Що таке фізична модель бази даних? Логічна?

Логічна модель — опис сутностей, атрибутів і зв'язків незалежно від СУБД.

Фізична модель — реалізація логічної моделі з урахуванням конкретної СУБД, типів даних і індексів.

14. Який взаємозв'язок між таблицями БД та програмними класами?

Зазвичай кожна таблиця відповідає одному класу, а стовпці — його властивостям (ORM-відповідність).

Висновок

У ході лабораторної роботи було засвоєно основні принципи роботи з системою контролю версій **Git**. Було створено репозиторій, гілки, виконано коміти та злиття. На практиці відпрацьовано вирішення конфліктів і перегляд історії змін. Отримані навички дозволяють використовувати Git для організації командної роботи та керування версіями проєктів.